

プログラム言語論

亀山幸義

筑波大学 情報科学類

No. 4

目次

1 OCaml における型

2 高階関数

3 第 2 回演習問題

真理値型 (bool)

```
let f1 x y = (x mod y) == 0;;
```

```
f1 5 3;; (* = false *)
```

```
f1 6 3;; (* = true *)
```

```
let f2 x y = (f1 x y) && (x != y) && (y > 1);;
```

```
f2 9 3;; (* = true *)
```

```
f2 9 1;; (* = false *)
```

```
let f3 x y = (f1 x y) || (f1 y x);;
```

```
f3 2 8;; (* = true *)
```

整数型 (int)

```
let g1 x = 3 + x / 2 * 3;;  
g1 5 ;; (* = 9 *)  
g1 -5 ;; (* エラー *)  
g1 (-5) ;; (* = -3 *)  
max_int;; (* = 整数の最大値 *)
```

割り算は、整数上の (小数点以下を切捨て) 割り算

「-」は通常は2項演算子 (a-b) を表すので、負の数は括弧が必要.

例: f - 3 でなく f (-3)

浮動小数点数の型 (float)

```
let h1 x y = x *. 3.14 -. y;;  
h1 5 3;; (* エラー *)  
h1 2. 3.14;; (* = 3.14 *)
```

```
float 2;; (* = 2. *)  
floor 3.14;; (* = 3. *)  
truncate 3.14;; (* = 3 *)
```

浮動小数点数と整数は異なる型なので、まぜて使えない。
浮動小数点数に対する演算名には「.」がつく。

文字列型 (string)

```
"ab" ^ "cd" ^ "ef";; (* = "abcdef" *)  
string_of_int (3 * 5);; (* = "15" *)  
string_of_float (sqrt 3.);;  
(* = "1.73205080757" *)  
  
print_endline ("a" ^ (string_of_int 3) ^ "b");;  
(* a3b が印刷される. 返り値は () *)
```

組型 (tuple)

組は、決まった個数の集まりで、組の要素は違う型でよい.

```
let mkpair x y = (x,y);;  
let get_x (x,y) = x;;  
let rotate (x,y) =  
    let c = cos (Float.pi /. 4.) in  
    let s = sin (Float.pi /. 4.) in  
    (x*.c-.y*.s, x*.c+.y*.s);;  
rotate (1.5, 2.3);;  
(* 反時計回りに  $\pi/4$  回転した点 *)  
let dist (x,y) = sqrt (x*.x +. y*.y);;  
dist (1.,2.);; (* 原点までの距離 *)
```

リスト型 (list) (1)

リストは、不定個の集まりで、要素は同じ型.

```
let list1 = [10; 20; 30];;  
List.hd list1;; (* = 10 *)  
List.tl list1;; (* = [20; 30] *)  
3 :: list1;; (* = [3; 10; 20; 30] *)  
1.2 :: (2.5 :: []);; (* = [1.2; 2.5] *)  
1 :: (2.5 :: []);; (* エラー *)
```


リスト型 (list) (2)

以下のものは (内部的には) 同一のリスト

```
[10; 20; 30]
```

```
10 :: (20 :: (30 :: []))
```

```
10 :: 20 :: 30 :: []
```

(*** は右結合 :: ***)

リスト型 (list) (3)

リストを使う (パターンマッチ)

```
let f1 x =  
  match x with  
  | [] → 0  
  | h::t → h;;
```

```
f1 [];; (* = 0 *)
```

```
f1 [10; 20];; (* = 10 *)
```

```
let rec f2 x =  
  match x with  
  | [] → 0  
  | h::t → 1 + (f2 t);;
```

```
f2 [10; 20; 30];; (* = 3 *)
```

```
f2 ["ab"; "cdefg"];; (* = 2 *)
```

関数型 ($\rightarrow$)

$A \rightarrow B$ (OCaml では $A \rightarrow B$): 「型 A の値を引数として型 B の値を返す関数」の型

```
fun x  $\rightarrow$  x + 1      : int  $\rightarrow$  int
fun x  $\rightarrow$  "abc"      : int  $\rightarrow$  string
fun x  $\rightarrow$  if x then 0 else 1 : bool  $\rightarrow$  int
fun x  $\rightarrow$  (fun y  $\rightarrow$  x + y)   : int  $\rightarrow$  (int  $\rightarrow$  int)
```

目次

1 OCaml における型

2 高階関数

3 第2回演習問題

再帰関数を手軽に書く (1)

リストに関する再帰関数をもっと楽に書こう.

```
let list1 = [10; 20; 30];;  
let g1 x = x + 2;;  
let g2 x = sqrt (float x);;  
  
List.map g1 list1;;  
(* = [12; 22; 32] *)  
open List;;  
map g2 list1;;  
(* = [3.1...; 4.4...; 5.4...] *)  
  
let list2 = [3.14; 5.8; -2.5];;  
let g3 x = (truncate x) * 2;;  
map g3 list2;;  
(* = [6; 10; -4] *)  
  
map g3 list1;; (* エラー *)  
map g2 list2;; (* エラー *)
```

再帰関数を手軽に書く (2)

リストに関する再帰関数をもっと楽に書こう.

```
let list1 = [10; 20; 30];;  
let h1 x y = x + y * 2;;  
let h2 x y = x * 3 + y;;  
  
open List;;  
fold_left h1 5 list1;;  
(* = h1 (h1 (h1 5 10) 20) 30 = 125 *)  
fold_left h2 7 list1;;  
(* = h2 (h2 (h2 7 10) 20) 30 = 369 *)  
fold_right h2 list1 7;;  
(* = h2 10 (h2 20 (h2 30 7)) = 187 *)
```

再帰関数を手軽に書く (3)

```
(* length *)
fold_left (fun acc h → acc + 1) 0 list1;;

(* sum *)
fold_left (fun acc h → acc + h) 0 list1;;

(* reverse *)
fold_left (fun acc h → h :: acc) [] list1;;

(* id *)
fold_right (fun h acc → h :: acc) [] list1;;
```

再帰関数を手軽に書く (4)

`map`, `fold_left`, `fold_right` は高階関数 (higher-order function):

```
map g1 [3;4;5];;  
fold_left h2 5 list1;;  
fold_right h2 list1 7;;
```

(ここで、`g1` や `h2` は関数)

再帰関数を手軽に書く (4)

`map`, `fold_left`, `fold_right` は高階関数 (higher-order function):

```
map g1 [3;4;5];;  
fold_left h2 5 list1;;  
fold_right h2 list1 7;;
```

(ここで、`g1` や `h2` は関数)

式 `map g1 [3;4;5]` の型付け:

```
map           : (int → int) → (int list → int list)  
g1            : int → int  
map g1        : int list → int list  
map g1 [3;4;5] : int list
```

まとめ

ラムダ計算と型:

- ▶ 「関数」は始域と終域が備わっているはず
- ▶ 基本的なラムダ計算に始域・終域はない。
- ▶ $\Rightarrow$ 型をいれて、始域・終域の整合性をチェック
- ▶ $f : A \rightarrow B$ かつ $e : A$ ならば $f e : B$
- ▶ $(\lambda x. x + 1) "abc"$ といった式を排除

型の検査:

- ▶ 関数適用 $f e$ の型検査だけなら簡単
- ▶ 高階関数などがあると、直感だけではやっていけない。
- ▶ $\Rightarrow$ 型付きラムダ計算と型検査・型推論 (来週)

目次

① OCaml における型

② 高階関数

③ 第2回演習問題

演習問題 (1)

再帰を使わずに (`map`, `fold_left`, `fold_right` は使ってよい) を以下の関数を実装しなさい。

▶ 関数 `g1`:

浮動小数点数 (`float` 型の値) からなるリストを引数として、その平均値を `float` 型の要素として返す。(引数が空リストのときは、どんな `float` 型の要素を返してもよい。 `g2` も同様)

▶ 関数 `g2`:

浮動小数点数 (`float` 型の値) からなるリストを引数として、その分散 (各要素と平均値の差の二乗の総和を要素数で割ったもの) を `float` 型の要素として返す。

例: `g2 [1.0;2.0;2.0;4.0;4.0;5.0] = 2.0`

▶ 関数 `g3`:

`g2` と同じ引数を取り、「平均値+標準偏差」以上の要素数を `int` 型の要素として返す。(標準偏差は分散の平方根である) (引数が空リストのときは、0 を返しなさい。)

例: `g3 [1.0;2.0;2.0;4.0;4.0;5.0] = 1`

演習問題 (2)

▶ 関数 g4:

「整数をもらって整数を返す関数」たちからなるリストを引数として、その関数たちをそれぞれ 3 に適用した値 (関数に、3 を引数として渡して計算した結果) のリストを返す関数。(引数が空リストのときは、空リストを返しなさい。)

例: `g4 [fun x -> x+1; fun x -> x*2] = [4;6]`

▶ 関数 g5:

`g4` と同じ引数を得て、その関数たちをリストの左にある関数から順番に合成したもの ($f_1 \circ f_2 \circ \dots \circ f_n$) を、3 に適用した値を返す関数。(引数が空リストのときは、3 を返しなさい。)

例: `g5 [fun x -> x+1; fun x -> x*2] = 7`

`map` 等を使うときは、関数名の頭に `List.` をつけて `List.map` とするか、あるいは `open List;;` としてから `map` を使うこと。